Time, causality and global state

Distributed Systems

Alberto Lafuente, Mikel Larrea
UPV/EHU

Contents

1 Introduction
2 Physical time
- 2.1 External synchronization
- 2.2 Internal synchronization
- 2.3 Compensation of deviations
- 2.4 Examples

3 Logical time and ordering of events
- 3.1 Event model
- 3.2 *Lamport*'s logical clocks
- 3.3 Vector clocks

4 Global state and consistency
- 4.1 System model
- 4.2 Determination of consistent global states
- 4.3 Algorithm of *Chandy* and *Lamport*

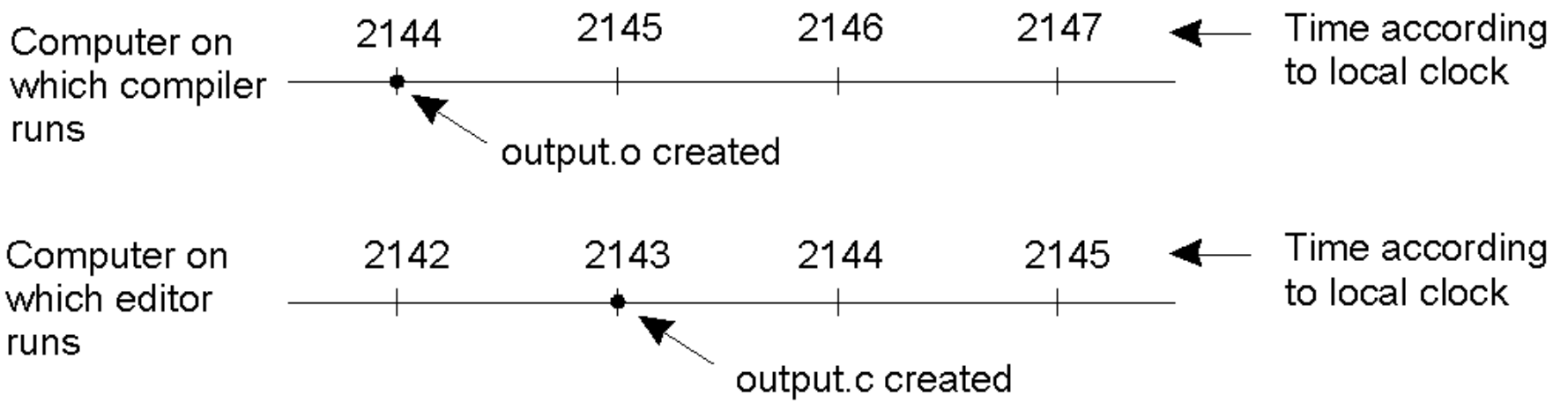
1 Introduction

- In a distributed system the *global state* is distributed among the nodes
  - standard clock in each node
  - communication between nodes: delays
  - each node has a subjective view of the global state
- Examples:
  - at time t_1 a message is sent from node *A* to node *B*, which receives it at time t_2, with $t_2 < t_1$
  - establish with absolute precision the amount of deposits in all bank accounts of all banks in the world at a given time

1 Introduction

- Solution proposal: single precise clock + dedicated network to transmit its signal without delays
  - not practical (cost), unfeasible (Internet)
- Solution: distributed time
  - each node has its own clock (local physical time)
  - clocks are imprecise: they need to be adjusted periodically to a reference physical time
- On the other hand, the consistent management of the global state requires at least ordering the events produced by the nodes (causality)
  - logical time

Clock synchronization

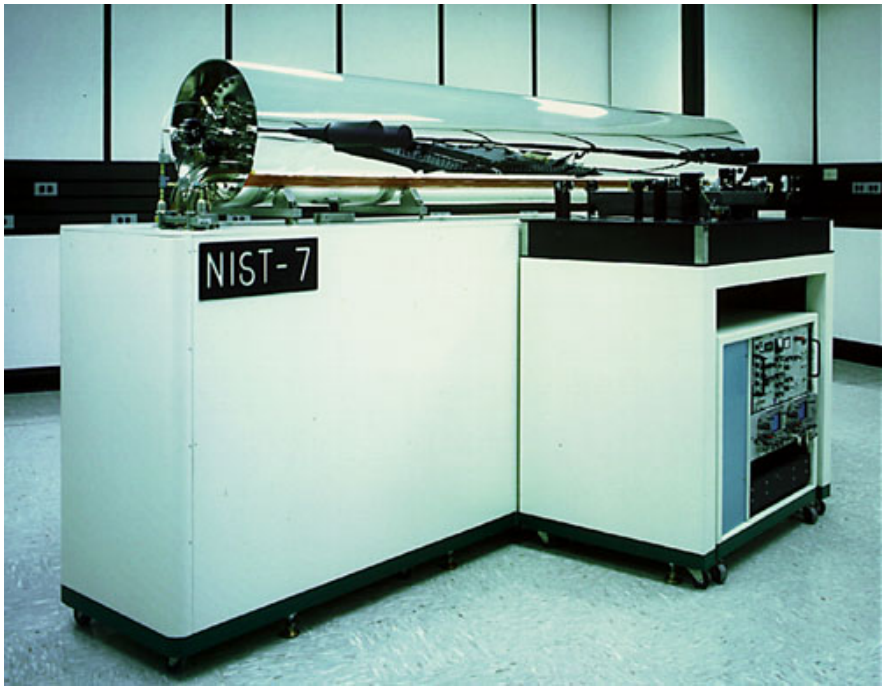


- When each machine has its own clock, an event that occurred after another event may nevertheless be assigned an earlier time

2 Physical time

- Computer clocks are made of quartz
  - their frequency of oscillation varies with temperature
  - drift: ~10^{-6} (90ms in one day, 1s each 11,6 days)
- Atomic clocks: great precision, very expensive
  - drift: ~10^{-13} (9ns in one day, 1s each 300.000 years)
  - cost: $50.000 - $100.000 !!!
- Types of synchronization:
  - know the precise moment in which an event occurs in a node: external synchronization
  - measure the interval between two events produced in different nodes (for example, sending and receiving a message), using the local clocks of each node: internal synchronization
  - the internal one does not imply the external, but vice versa

Atomic clocks



NIST-7 (1993)
Drift: 5×10^{-15}



NIST-F1 (2005)
Drift: 5×10^{-16}

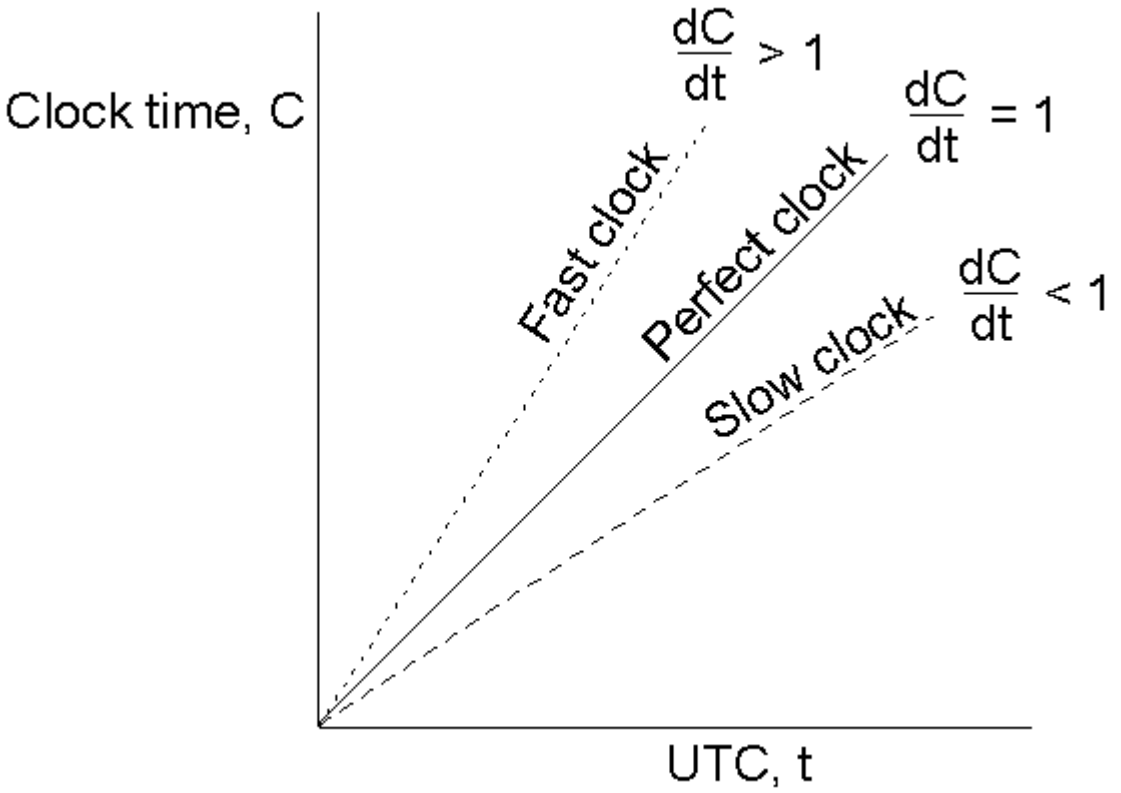
2 Physical time

- Definitions:
  - Mean solar second (astronomical): 1/86.400 of the period of rotation of the Earth
    - despite being perfectly valid for everyday situations, the Earth does not rotate at a constant speed (it slowly loses speed), so it does not serve as a reference
  - Atomic second (*IAT*, 1967): 9.192.631.770 transition periods in a Cesium-133 atom. Atomic clocks measure this time
    - drift of $3*10^{-8}$ with the solar second (~1*s* in a year)
  - Coordinated Universal Time (*UTC*): measured in atomic seconds, synchronized with astronomical time (difference > 900 *ms* $\Rightarrow$ insertion of 1*s*)

2 Physical time

- Definitions:
  - Reference physical time: normally *UTC*
  - Resolution: period between two updates of the local time register
    - must be less than the minimum time interval between two events produced consecutively in the node
  - Deviation (*offset*, *skew*, θ): difference between local time and reference physical time in an instant
  - Drift (δ): deviation per unit of time (what advances or delays the clock)
  - Precision (*accuracy*): maximum deviation that can be guaranteed when adjusting a clock

Clock synchronization algorithms



- The relation between clock time and *UTC* when clocks tick at different rates

2.1 External synchronization

- Synchronization: procedure by which the value of a clock is adjusted to a reference physical time with a pre-established precision
- *UTC* time is broadcast periodically by radio
  - propagation speed: ~$3*10^8$ *m/s* (1.000 km: ~3 *ms*)
  - variable speed: loss of precision
- Accuracy of commercial *UTC* receivers:
  - Land stations: 0,1 - 10 *ms*
  - Geostationary satellites (35.786 *km*): 0,5 *ms*
  - GPS satellites (20.200 *km*): 1 μs
- Uses: precise time service, accounting…

2.2 Internal synchronization

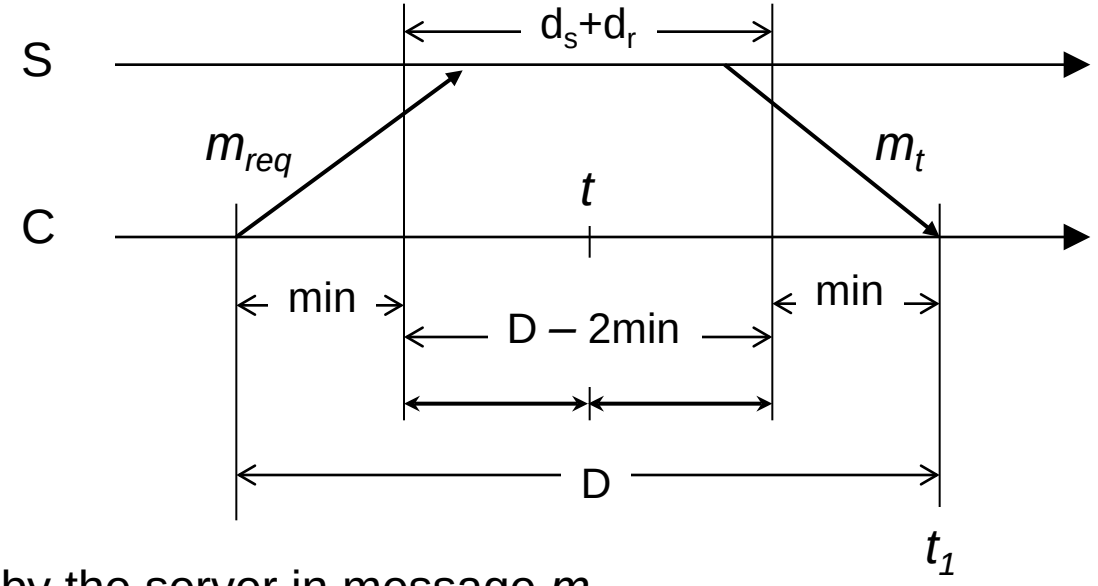
- For many applications it is more important to keep local clocks well synchronized with each other than to achieve high accuracy in external synchronization
  - it allows ordering events (causality)
- Solution proposal: *UTC* receiver in each node
  - not practical (cost)
- Solution: internal synchronization algorithms
  - centralized: based on a specific server
  - distributed: statistical

2.2.1 Centralized algorithms

- Idea:
  - use a time server that provides reliable time references (for example, a server that is periodically synchronized with *UTC*)
  - the client sends a request to the server (m_{req})
  - the server replies by sending the time reference in another message (m_t)
  - message delay: $min + d$
    - *min* is the minimum transmission time. It is constant and can be determined from the characteristics of the network
    - *d* is variable and depends on the load of the system and the network
- Example: *Cristian*'s algorithm

2.2.1 Centralized algorithms

Cristian's algorithm



$t(m_t)$: time returned by the server in message m_t

D: time since sending m_{req} and receiving m_t

min: minimum transmission time of a message

Assumption: the server assigns the reference $t(m_t)$ in the middle of the interval $D-2*min$ (instant t in the client). This gives the best precision

Deviation: $\theta = t - t(m_t) = t1 - D/2 - t(m_t)$

Precision = $D/2 - min$

2.2.1 Centralized algorithms

- Example of *Cristian*'s algorithm (3 requests):

Pet.	*D (ms)*	*t_1 (hh:mm:ss.ms)*	*$t(m_t)$ (hh:mm:ss.ms)*
(1)	*22*	*10:54:22.236*	*10:54:23.674*
(2)	*26*	*10:54:24.000*	*10:54:25.450*
(3)	*20*	*10:54:26.946*	*10:54:28.342*

  - What request should the client use? Calculate the precision and deviation obtained

    Precision = 10 *ms* Deviation = 1406 *ms*

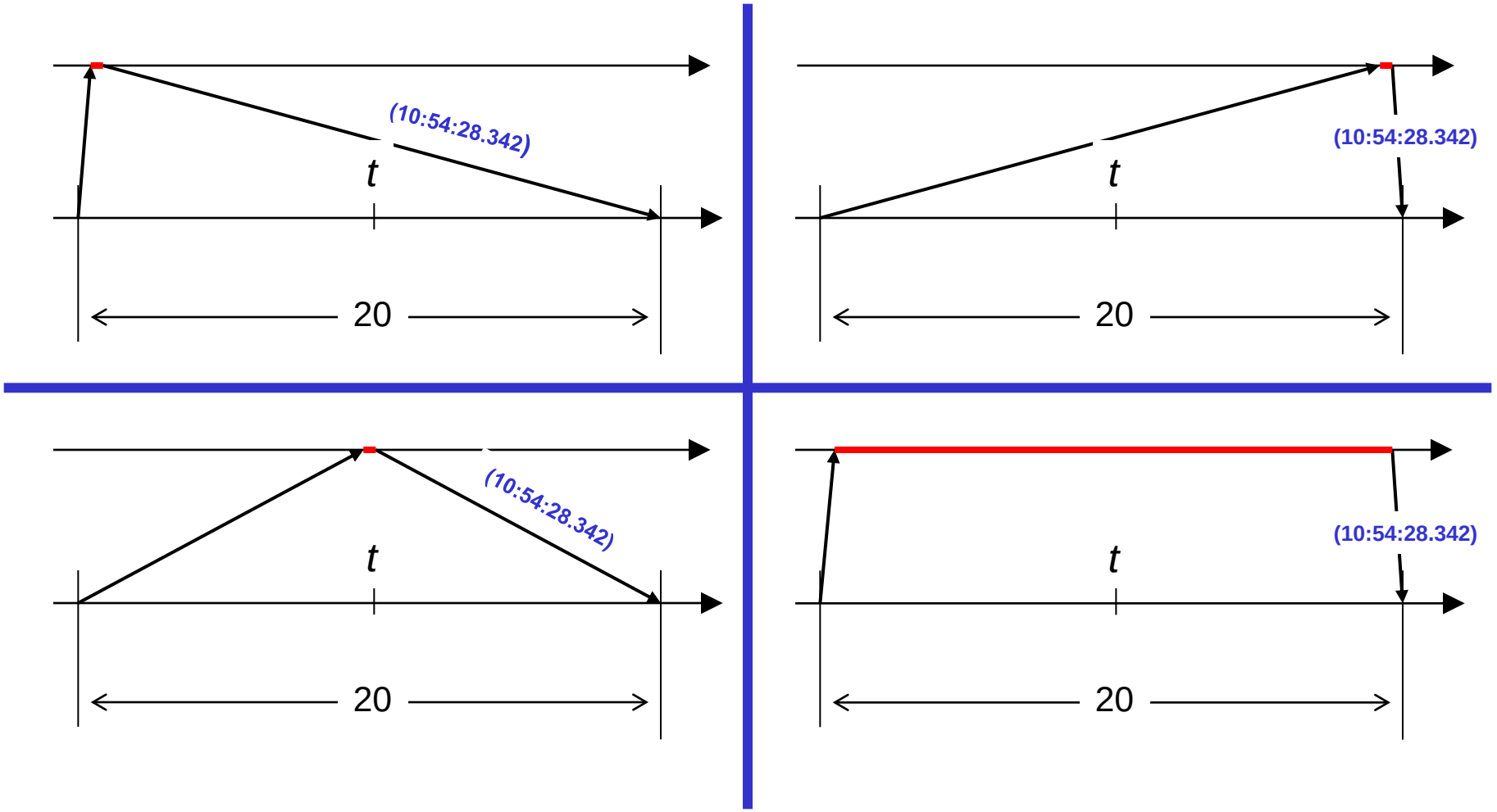
  - Knowing that the minimum transmission time is 7 *ms*, does the previous response change?

    Precision = 3 *ms*

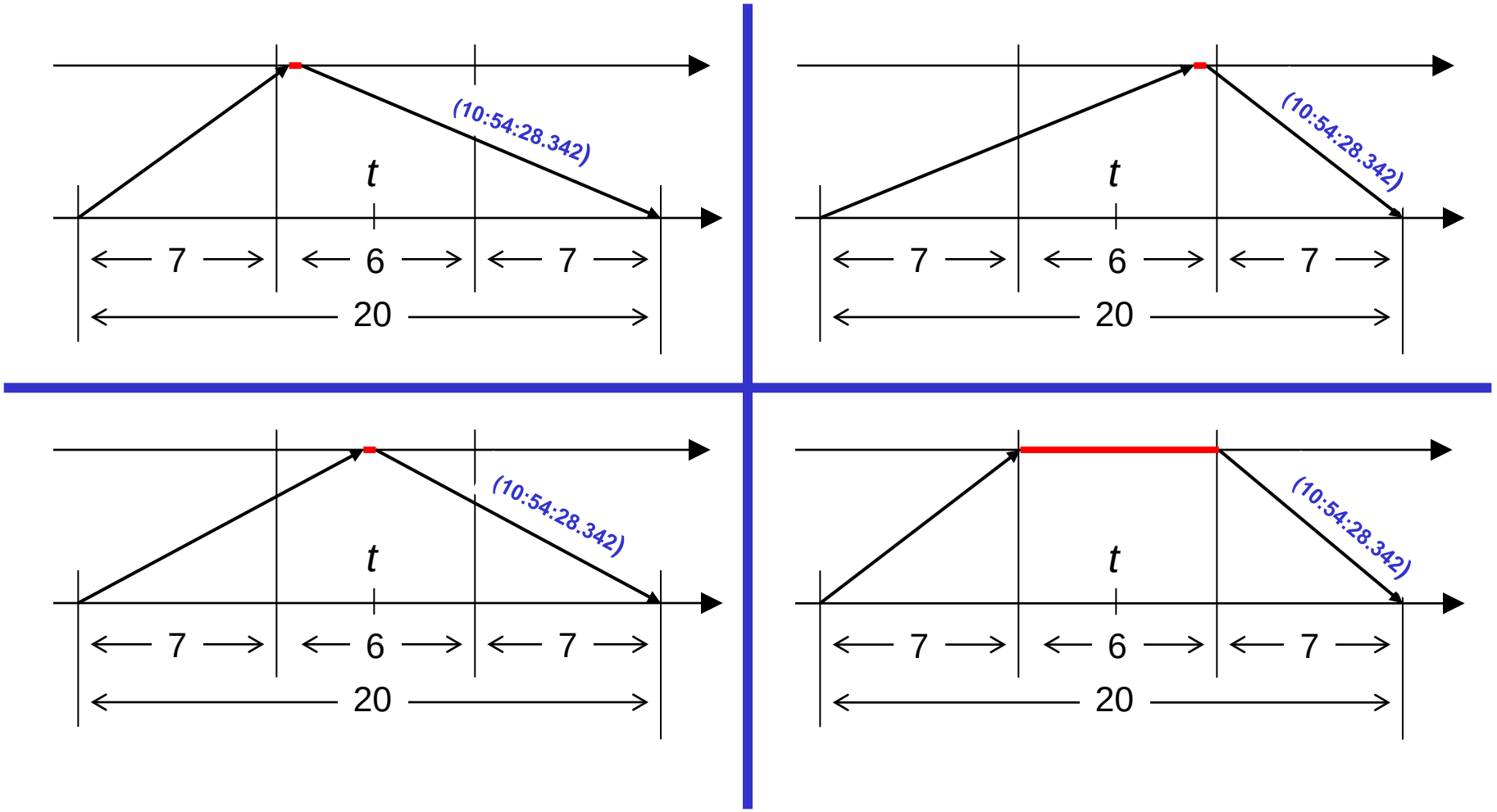
  - What is needed to obtain a precision of 2 *ms*?

    Being *min* = 7 *ms*, a request with $D \leq 18$ *ms*

2.2.1 Centralized algorithms



2.2.1 Centralized algorithms



2.2.2 Distributed algorithms

- Statistical principle of distributed algorithms:
  - the deviations of local times tend to compensate each other around an average time that can be taken as a reliable reference for internal synchronization
- Example: the *Berkeley* algorithm
  - the coordinator request the times to the rest of nodes
  - the coordinator calculates an average time based on the responses obtained, excluding spurious responses and those that do not arrive in time

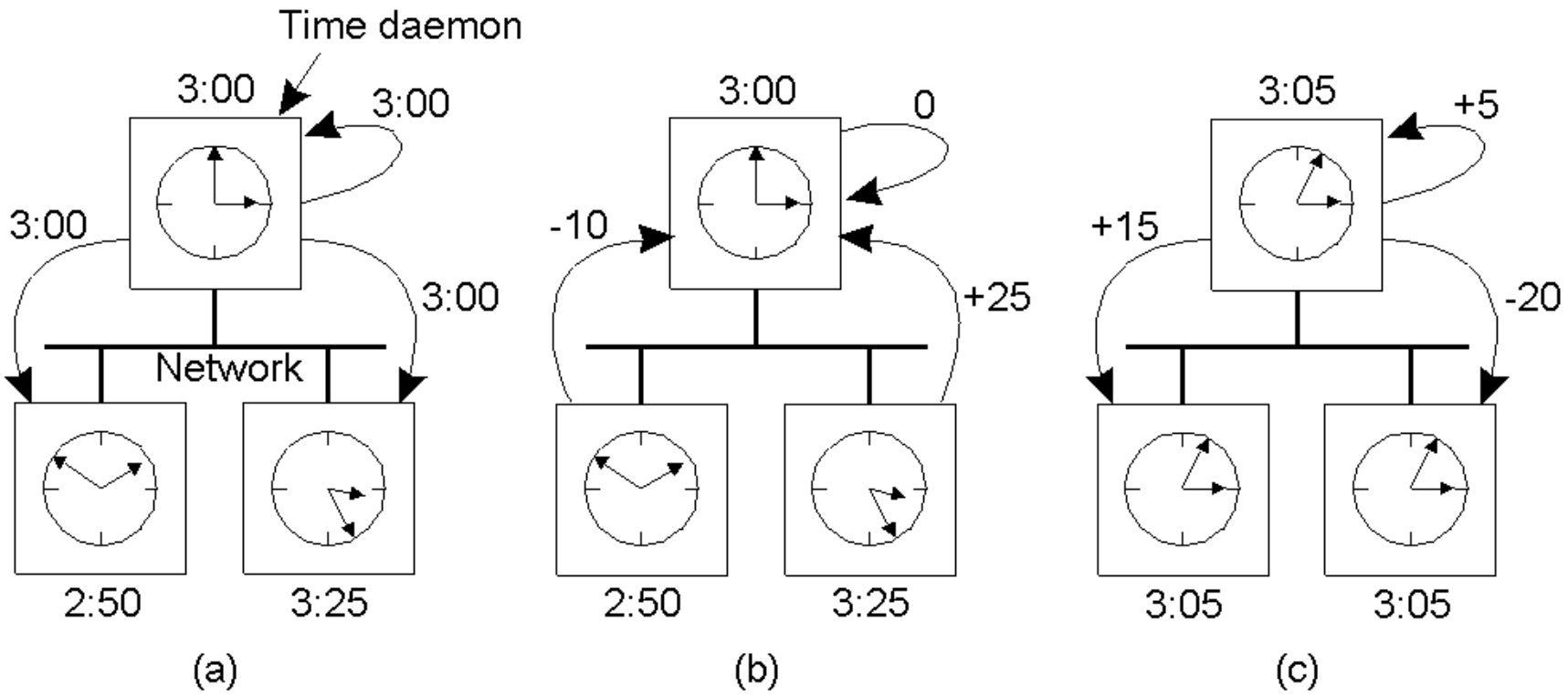
$$T = \Sigma(t(m_i) - D_i/2) / N$$

  - the coordinator sends to each node its deviation with respect to the average, in order to make the adjustment

$$\theta_i = t(m_i) - D_i/2 - T$$

  - if the coordinator fails, a new coordinator is elected

The Berkeley algorithm



a) The time daemon asks all the other machines for their clock values

b) The machines answer

c) The time daemon tells everyone how to adjust their clock

2.2.2 Distributed algorithms

- Example of the *Berkeley* algorithm:

	Node	*D (ms)*	*t (hh:mm:ss.ms)*	***Deviation***
	N_1 *(coord.)*	*0*	*10:54:23.118*	***-948 ms***
	N_2	*22*	*10:54:22.236*	***-1842 ms***
	N_3	*26*	*10:54:24.000*	***-79 ms***
Excluded!	N_4	*190*	*10:41:46.179*	***-757983 ms***
	N_5	*20*	*10:54:26.946*	***+2870 ms***

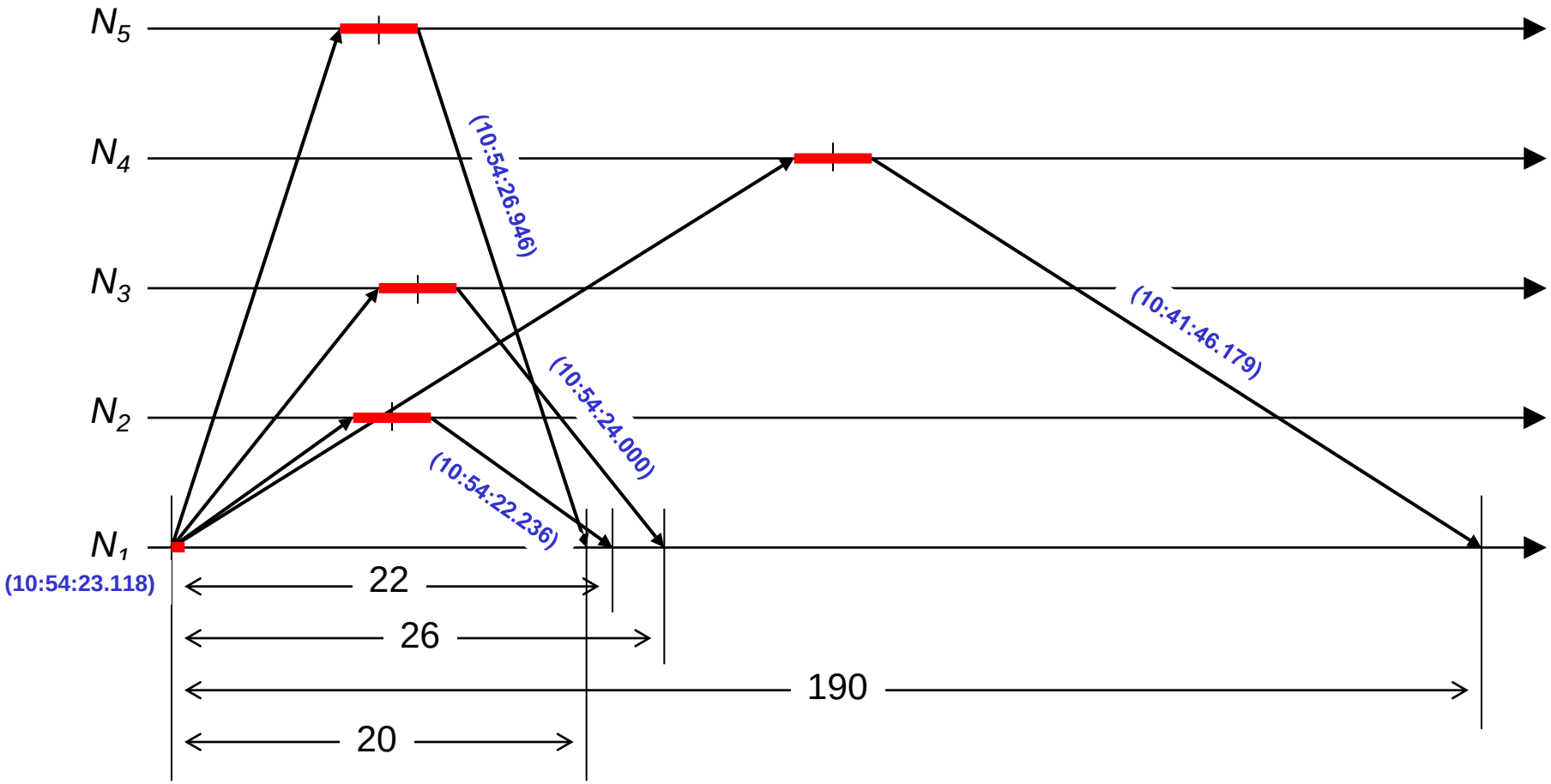
  - Calculate the average time for synchronization and the deviation to send to each node

    Average time = 10:54:24.067

  - Using *Cristian*'s method, calculate the precision in the adjustment of each node when synchronized (*min* is unknown)

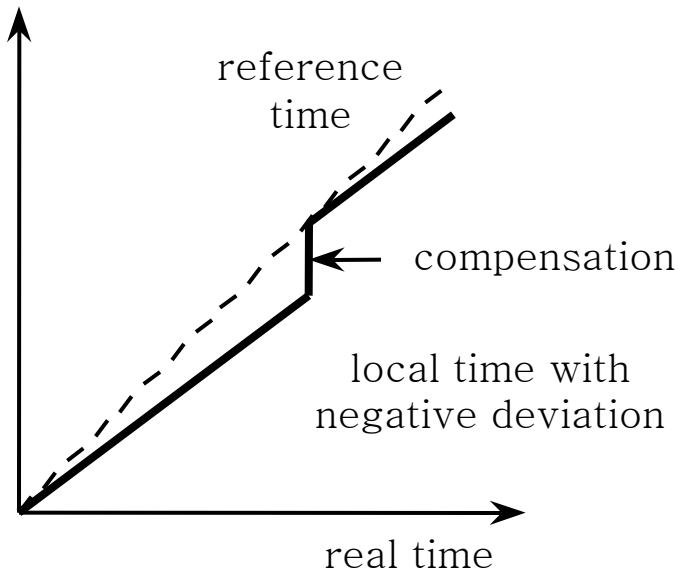
    ***D*/2**

2.2.2 Distributed algorithms



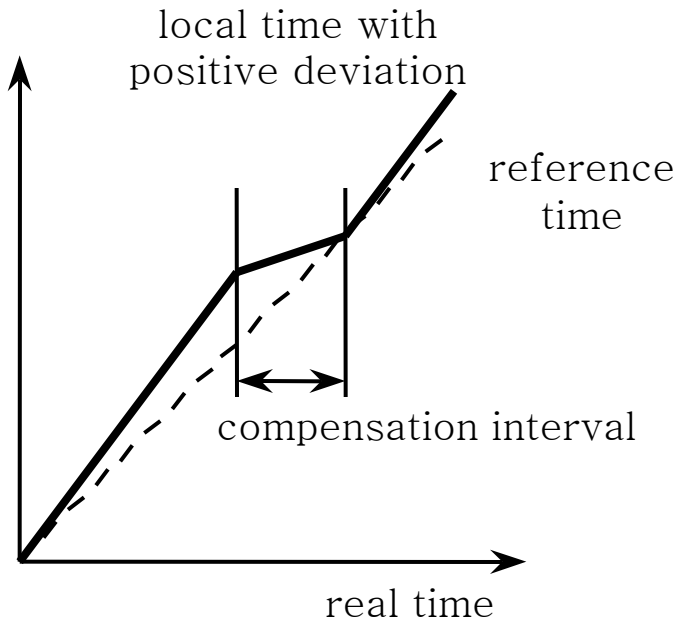
2.3 Compensation of deviations

- Fundamental property: <u>increasing monotonicity</u>



(a)

Negative deviation

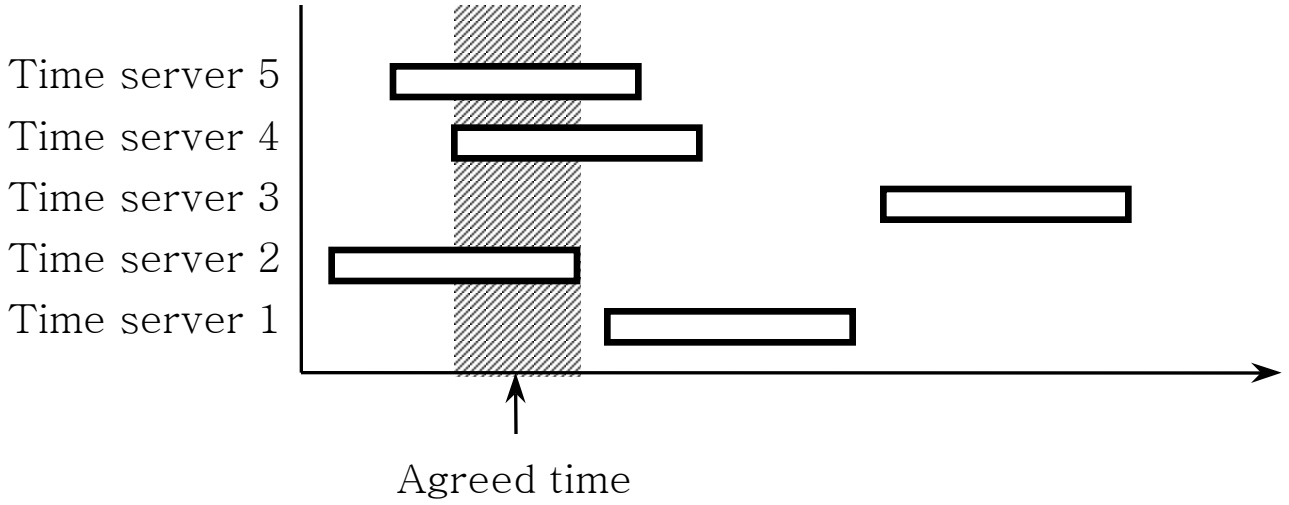


(b)

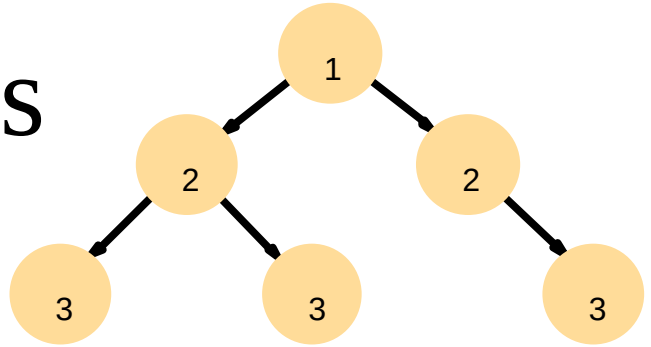
Positive deviation

2.4 Examples

- *Distributed Time Server (DTS)*
  - distributed algorithm
  - time servers: some of them synchronized with *UTC*
  - time clerks: clients



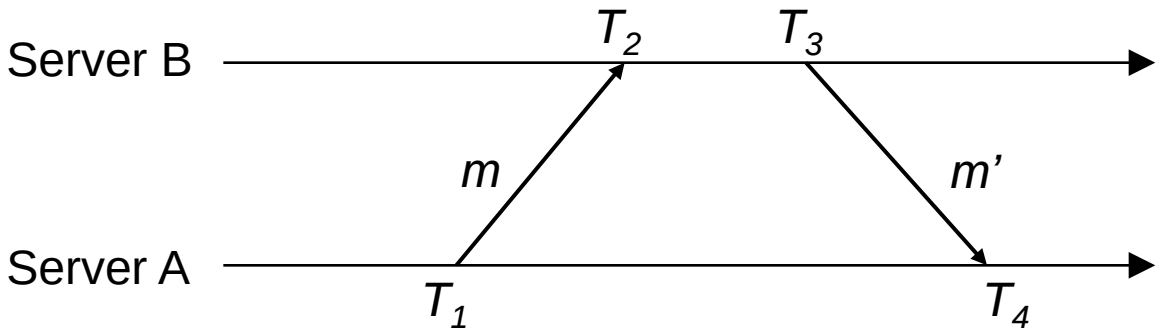
2.4 Examples



- *Network Time Protocol (NTP)*
  - Internet standard
  - provides redundant synchronization with *UTC*
  - structured in levels (*strata*) of time servers
    - primary servers (*strata* 1): reliable *UTC* references
    - level 2 servers (*strata* 2): synchronized with primaries
    - ...
  - operating modes:
    - local network with adequate support: *multicast* mode
    - higher precision: procedure call mode
    - better internal synchronization: symmetric mode
  - the server network can be configured

2.4 Examples

- *Network Time Protocol (NTP):* symmetric mode



$T_2 = T_1 + t + \theta$
$T_4 = T_3 + t' - \theta$

Hypothesis: A delayed with respect to B (θ)

$t, t' \geq 0$ are the transmission times of messages m and m'

$d_i = t + t' = T_2 - T_1 + T_4 - T_3 = (T_4 - T_1) - (T_3 - T_2)$

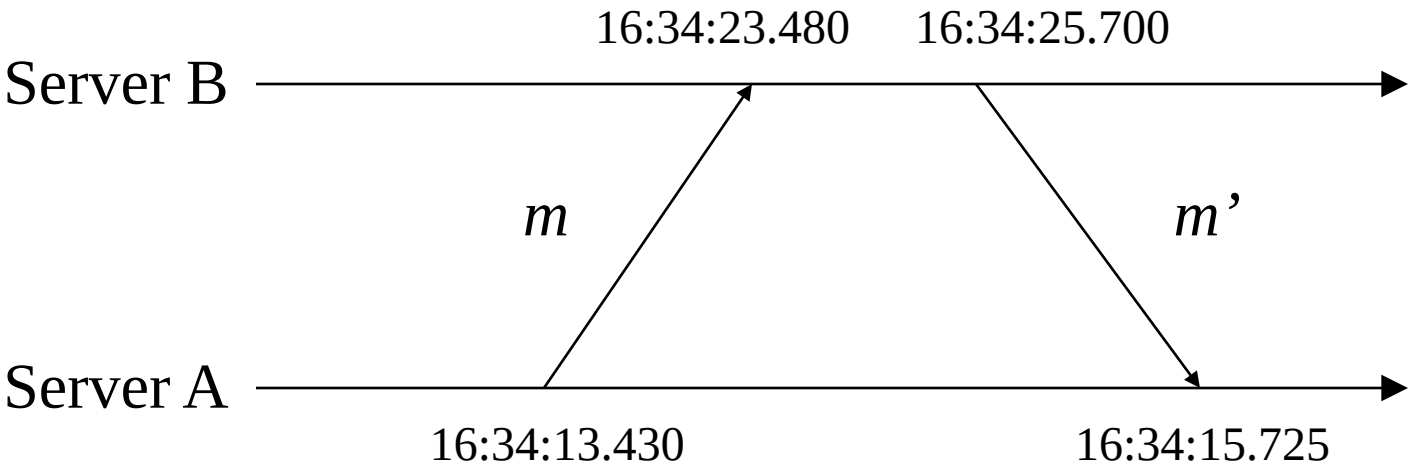
$\theta = \theta_i + (t' - t) / 2$, where $\theta_i = (T_2 - T_1 + T_3 - T_4) / 2$

$\theta_i - (d_i / 2) \leq \theta \leq \theta_i + (d_i / 2)$

θ_i deviation estimate; $(d_i / 2)$ precision

2.4 Examples

- *Network Time Protocol (NTP):* exercise



  - Deviation estimate?
  - Precision? **$t + t' = 0.075$, $\theta_i = 10.013$**

$\theta = 10.013 \pm 0.038$ (9.975 .. 10.051)

3 Logical time and ordering of events

- Sometimes the precision obtained when synchronizing the clocks does not allow us to use the physical time to order the events of the different nodes of a distributed system
  - it is always possible to order the events of the same node (if the monotonicity of the clock is respected)
  - however, causality relationships between events of different nodes can be distorted
- Many applications only require ordering events (and not so much knowing the exact moment when they occurred)

3.1 Event model

- System model: set of processes that communicate only by message passing

  send (p_i, *message*)

  receive (p_j, *message*)

  p_i and p_j are the sender and receiver of the message

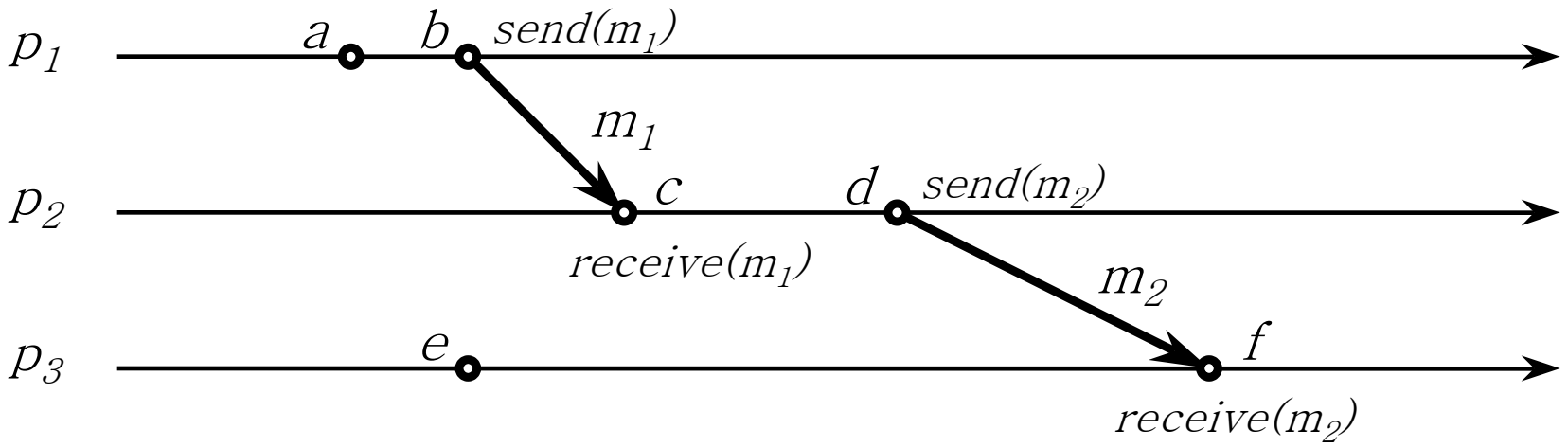
- Simplification: one process per node/machine
- Each process generates a sequence of events. We identify three types of events:
  - sending a message (when executing *send*)
  - receiving a message (when executing *receive*)
  - local/internal events (rest of events, without communication)

3.1 Event model

- To order the events of the same process it would be enough to associate each event *x* with the local time *T(x)* (if the resolution is sufficient)
- It is said that there exists a causality relationship between two events *x* and *y* of the system ($x \rightarrow y$, "*x happened before y*") if:
  1. *x* and *y* are events of the same process and $T(x)<T(y)$
  2. *x* and *y* are the events *send(m)* and *receive(m)* of the same message *m*
  3. There exists another event *z* such that $x \rightarrow z$ and $z \rightarrow y$ (transitive closure of the relationship)

3.1 Event model

- If there is no causality relationship between two events, we say that they are concurrent: $x \,||\, y$



$$a \rightarrow b \quad b \rightarrow c \quad a \rightarrow c \quad a \rightarrow f \quad a \,||\, e \quad e \,||\, d$$

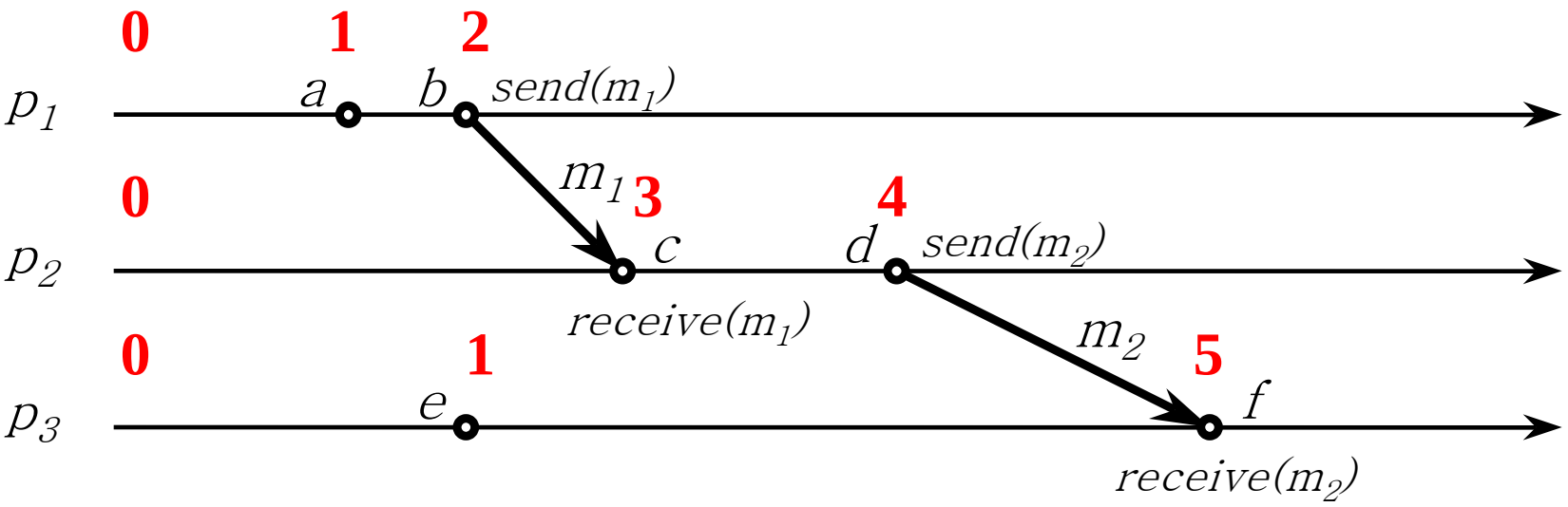
3.2 *Lamport*'s logical clocks

- Proposed to indicate causality relationships. Each process p_i has its local logical clock C_i, to associate timestamps to its events (a simple asynchronous counter is enough)
- Algorithm:
  - Initially, $C_i = 0, \ \forall i$
  - Before a local event or sending a message at p_i: $C_i = C_i + 1$
  - When p_j sends a message m to p_i, p_j includes the value of its logical clock in the message, C_m. When receiving that message, p_i updates its local clock as follows:

$$(1)\ C_i = max(C_i, C_m), \quad (2)\ C_i = C_i + 1$$

  - Issue: $C_i(x) < C_j(y)$ does not imply $x \rightarrow y$

3.2 *Lamport*'s logical clocks



$a \rightarrow b \quad b \rightarrow c \quad a \rightarrow c \quad a \rightarrow f \quad a \,||\, e \quad e \,||\, d$

3.3 Vector clocks

Guarantee: $V_i(x) < V_j(y) \Leftrightarrow x \rightarrow y$

- V: vector of N components (N=number of processes)
- $V_i[i]$: logical clock (local) of process p_i
- $V_i[j]$: last value that process p_i knows about the clock of process p_j

- Algorithm:
  - Initially, $V_i[j] = 0, \ \forall i, j$
  - Local event or sending a message at p_i: $V_i[i] = V_i[i] + 1$
  - When p_j sends a message m to p_i, p_j includes the value of its vector clock, V_m. When receiving that message, p_i updates its vector clock as follows:

    (1) $\forall k: V_i[k] = max(V_i[k], V_m[k])$, (2) $V_i[i] = V_i[i] + 1$

3.3 Vector clocks

- Definition:

$$V1 < V2 \Leftrightarrow \forall i\ V1[i] \leq V2[i] \text{ and } \exists j\ V1[j] < V2[j]$$

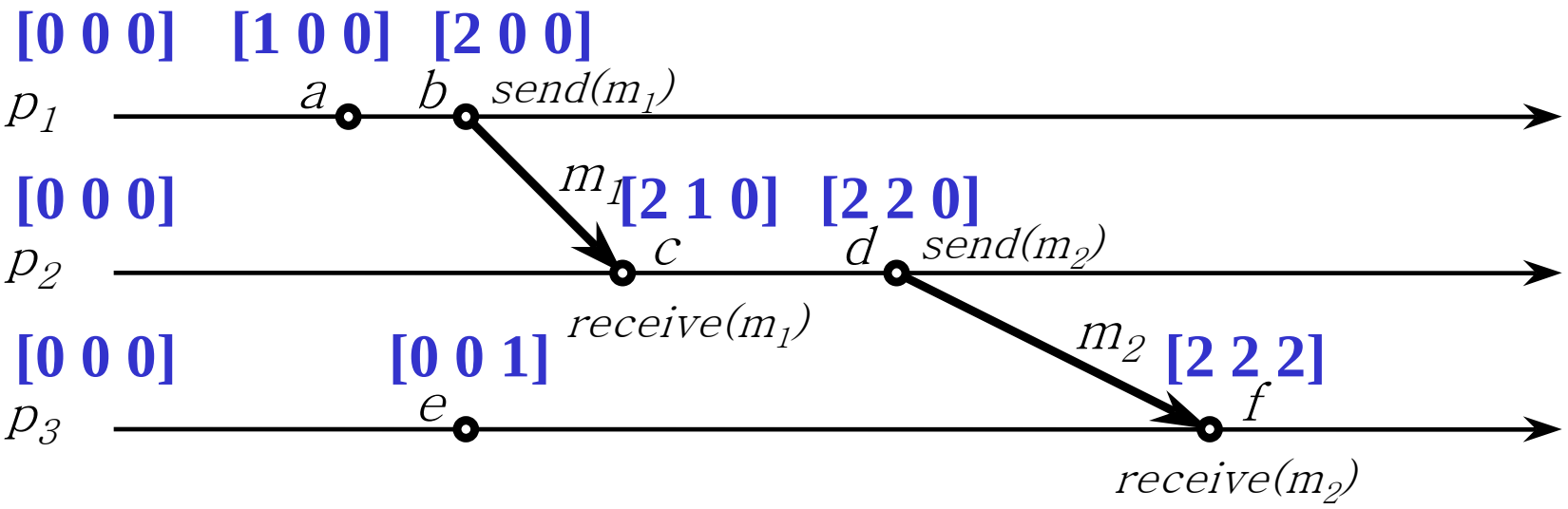
- There will be a causality relationship between two events *(x, y)* if and only if

$$V(x) < V(y) \text{ or } V(y) < V(x)$$

- Otherwise, the events are concurrent
- Vector clocks allow to determine causality relationships with total precision:

$$V_i(x) < V_j(y) \Leftrightarrow x \rightarrow y$$

3.3 Vector clocks



$$a \rightarrow b \quad b \rightarrow c \quad a \rightarrow c \quad a \rightarrow f \quad a \,||\, e \quad e \,||\, d$$

4 Global state and consistency

- System model:
  - N processes p_i (i = 1, 2, ..., N)
  - history(p_i) = $h_i = < e_i^0, e_i^1, e_i^2, ... >$
  - prefix of history $h_i^k = < e_i^0, e_i^1, ..., e_i^k >$
  - s_i^k: state of process p_i after the event e_i^k
  - s_i^0: initial state of process p_i
- Global state of the system: $S = (s_1, s_2, ..., s_N)$
  - Some global states can be “inconsistent”
- Cut: union of process history prefixes
  - $C = h_1^{c1} \cup h_2^{c2} \cup ... \cup h_N^{cN} = (c_1, c_2, ..., c_N)$

4 Global state and consistency

- Border of a cut: union of the last events

$$(c_1, c_2, ..., c_N) \Rightarrow (e_1^{c1}, e_2^{c2}, ..., e_N^{cN})$$

- A global state corresponds to each cut

$$(c_1, c_2, ..., c_N) \Rightarrow (s_1^{c1}, s_2^{c2}, ..., s_N^{cN})$$

- A cut C is consistent if for all pair of events (e, e') the following is true:

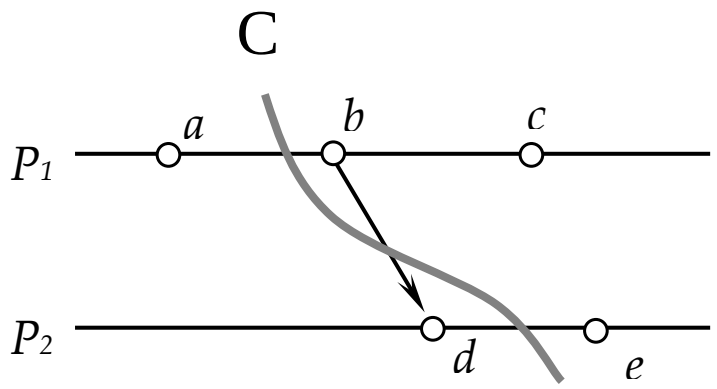
$$(e \in C) \wedge (e' \rightarrow e) \Rightarrow e' \in C$$

  - Otherwise, the cut C is inconsistent

- A consistent global state corresponds to each consistent cut

4 Global state and consistency

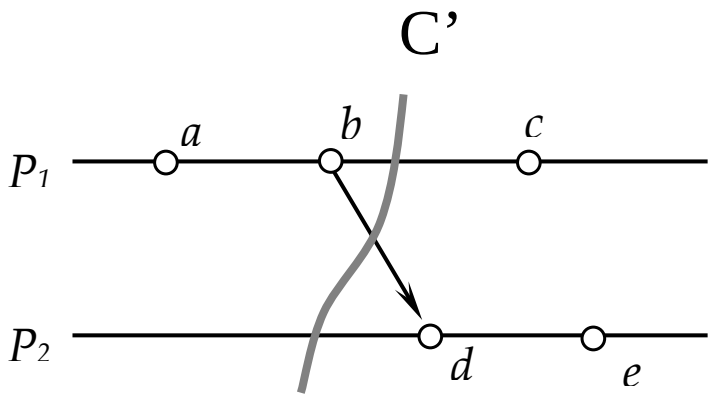
- Consistent cut: if the receiving event of a message is “inside” the cut, then the sending event of that message must be too



(a)

inconsistent

$d \in \mathrm{C}, b \rightarrow d, b \notin \mathrm{C}$



(b)

consistent

4 Global state and consistency

- To know if a cut is consistent, we can rely on vector clocks:

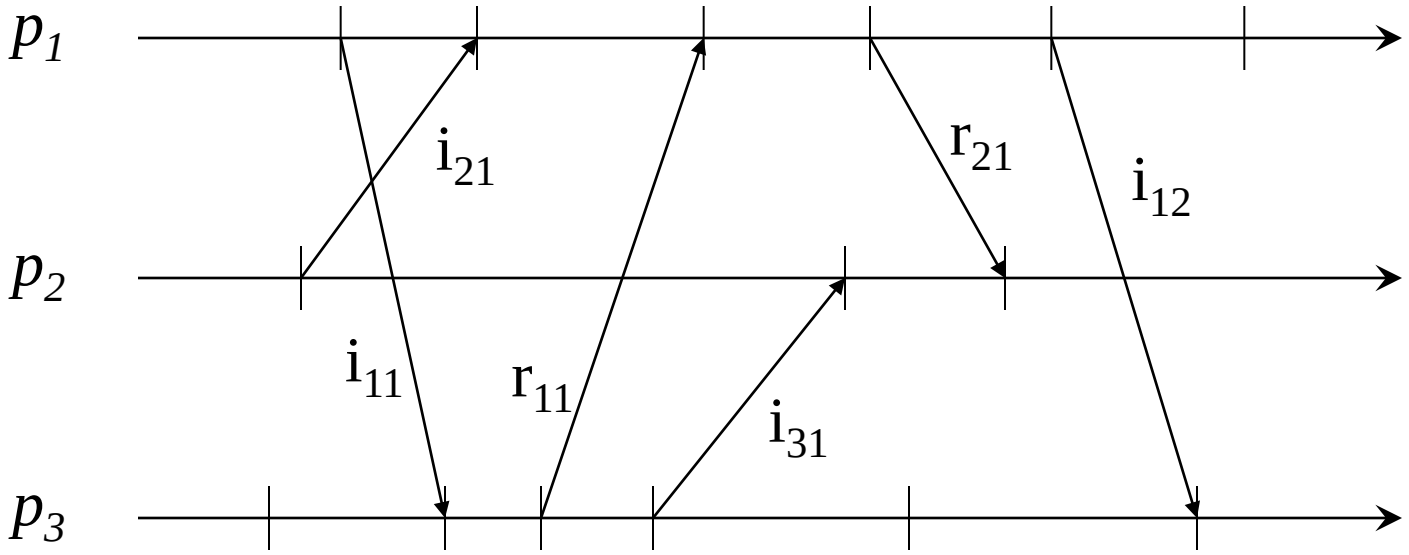
$$\forall i, j: V_i[i](e_i^{ci}) \geq V_j[i](e_j^{cj})$$

- Since each process has a partial view of the system, in order to get a consistent cut (and obtain its associated global state) processes must execute a distributed algorithm
  - Example: algorithm of *Chandy* and *Lamport*
- Usages: detection of deadlocks, establishment of recovery points of a system, distributed termination

4 Global state and consistency

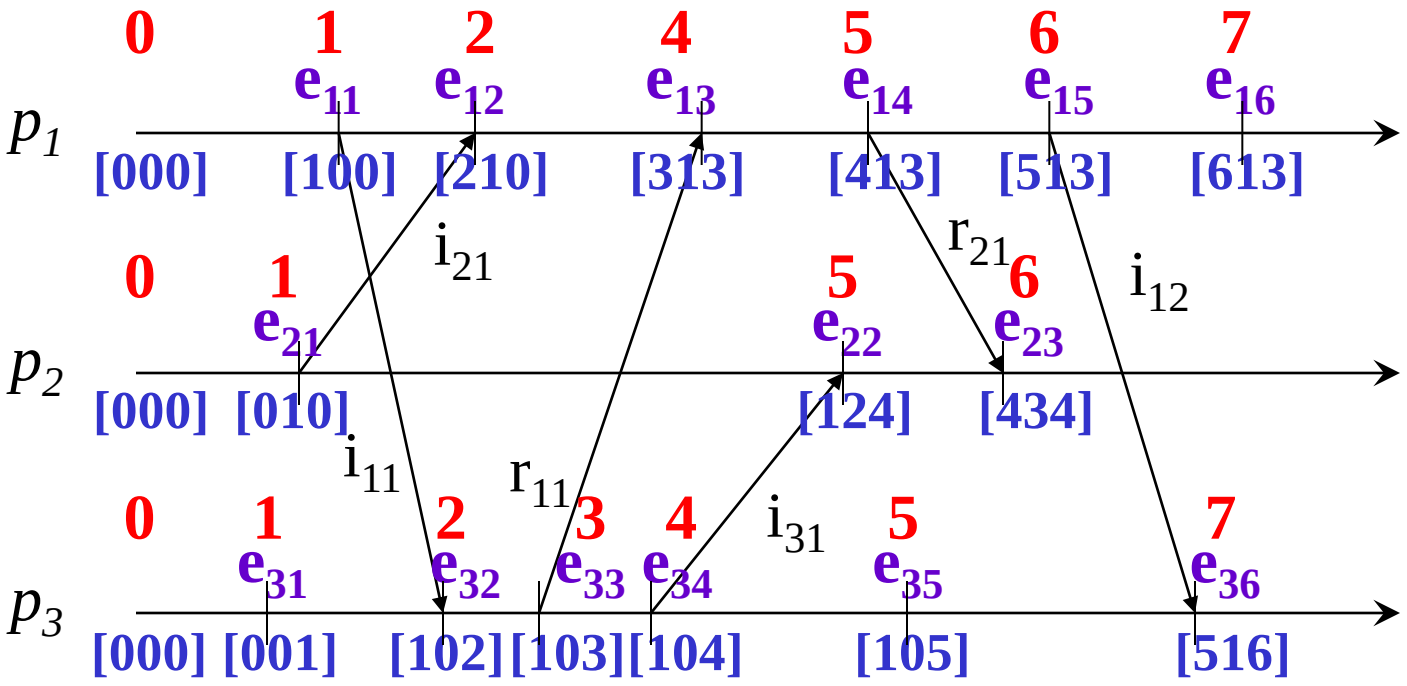
- *Snapshot* algorithm of *Chandy* and *Lamport*:
  1. process p_1 (the initiator of the snapshot) saves its state s_1 and broadcasts the message *SNAPSHOT* to P (the set of processes).
  2. Let process p_i receive the *SNAPSHOT* message the first time from some process p_j (p_j can be different from p_1). At that time, p_i saves its state s_i and forwards the *SNAPSHOT* message to P. The state of the channel c_{ji} is set to empty, and p_i starts logging the messages received on the channels c_{ki} (for all $k \neq j$).
  3. When p_i receives *SNAPSHOT* from p_k, then the computation of the state of the channel c_{ki} is complete. As soon as p_i has received *SNAPSHOT* from all the processes in P, the computation of the snapshot is terminated.

4 Global state and consistency



i: *invocation*
r: *reply*

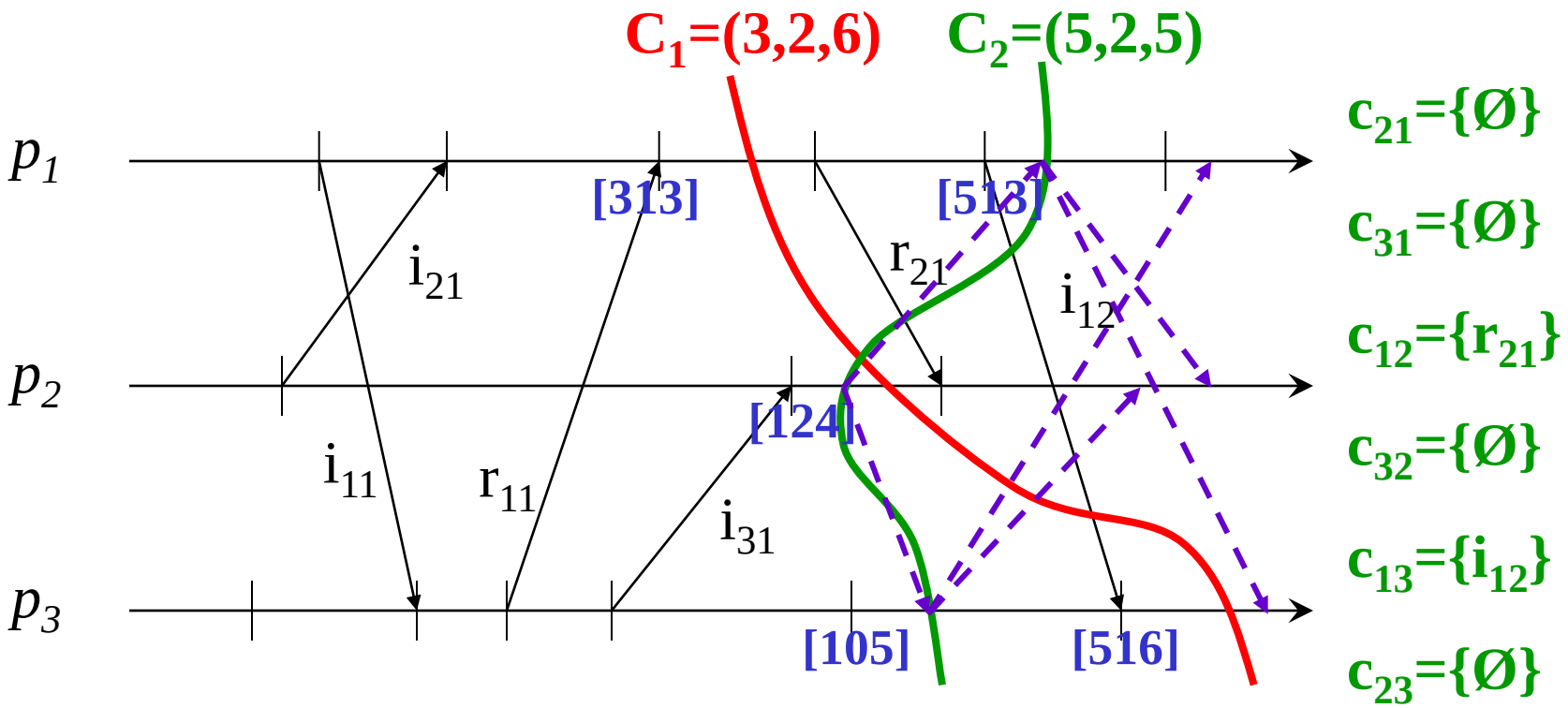
4 Global state and consistency



i: *invocation*

r: *reply*

4 Global state and consistency



i: *invocation*

r: *reply*